Part 1 Introduction

This experiment requires the implementation of four main projects: (1) iterative multiplier, (2) iterative divider, (3) modified Booth algorithm, and (4) three-input multiplier, all completed using the hardware description language Verilog. The first two are basic hardware arithmetic units; the third is an advanced version of the iterative multiplier, where the purpose of the modified Booth algorithm is to reduce the required computation cycles. Its algorithm and datapath are based on Figures 3 to 7 in the Lab1 handout, and both must be combined to achieve correct implementation. The fourth further extends the concept of the Booth multiplier, from the traditional multiplication of two numbers to the simultaneous multiplication of three numbers, requiring consideration of the bitwise operations of three operands. In addition, the experiment also includes the val/rdy interface, which serves as a data transfer protocol between modules to ensure correctness of data exchange and to avoid timing errors. In terms of performance requirements, the iterative multiplier and iterative divider have an upper bound of 33 cycles, the modified Booth multiplier must be completed within 17 cycles, while the three-input multiplier only requires verification of result correctness. Finally, unit tests and waveform analysis are used to verify the correctness and performance of the design.

Part 2 Design

(1) Iterative Multiplier

Datapath:

- Since the number of iterations is 32, a 5-bit counter is required. Ctr_done is used to determine whether the iteration is completed.
- Because the operation may involve signed and unsigned numbers, the input is processed in two's complement unsigned form, and then sign_reg = a[31]^b[31] records the result sign. Finally, the output uses sign_mux_sel to decide whether to restore result_reg with two's complement. This method allows only unsigned numbers to be handled during iteration, reducing Datapath complexity.
- a_reg[63:0]: equivalent to A in the algorithm, shifted left by 1 bit each iteration, a_shift_out = a_reg << 1.
- b_reg[63:0]: equivalent to B in the algorithm, shifted right by 1 bit each iteration, b_shift_out = b_reg >> 1.
- result_reg[63:0]: added with a_reg when b_reg[0] = 1.
- a_mux_sel, b_mux_sel: used to determine whether to shift or load the initial value.
- result_mux_sel: decides whether result_reg stores the result of result_reg + a_reg or the initial value.
- add_mux_sel: decides whether result_reg performs result_reg + a_reg or keeps its original value.

Control:

- IDLE: mainly responsible for initialization and loading, therefore mulreq_rdy = 1, indicating the module is ready to accept new requests. When mulreq_val = 1 is given, a_reg, b_reg, result_reg, and counter are loaded/reset, and the sign information sign_reg is latched. After loading is completed, FSM moves to RUN state to start the iteration calculation.
- RUN: mainly responsible for algorithm operation. Each iteration shifts a_reg left by one bit, b_reg right by one bit, and increments counter. When b_reg[0] = 1, result_reg ← result_reg + a_reg. When ctr_done = 1, meaning 32 iterations are completed, transfer to RESP state.
- RESP: mainly responsible for outputting the calculation result, and sets sign_mux_sel = 1 to ensure the correct signed number output. mulresp_val = 1 indicates the output result is valid. When mulresp_rdy = 1, FSM returns.

(2) Iterative Divider

Datapath:

- Since the operation may involve signed and unsigned numbers, the input first uses reg_sign_mux_sel to decide whether to take the absolute value, avoiding complicated control signals inside the loop. If it is signed, abs_a and abs_b are converted into positive numbers through two's complement, and div_sign = a[31]^b[31], rem_sign = a[31] record the signs of quotient and remainder respectively. Finally, according to sign_reg, decide whether quotient/remainder should be restored with two's complement.
- a_reg[64:0]: the lower 32 bits correspond to quotient, the higher 33 bits correspond to remainder.
- b_reg[64:0]：initialized as {1'b0, abs_b, 32'b0}, used for subtraction later.
- sub_out = a_shift_out - b_reg: executes subtraction according to algorithm flow, and each iteration will produce one quotient bit. If the subtraction result is non-negative, meaning one division is successful, then update the remainder with the difference and set the quotient bit to 1; if the result is negative, meaning division cannot proceed, then keep the shifted result and set the quotient bit to 0.
- sub_neg = sub_out[64]: used to determine whether the difference is less than 0 or greater/equal to 0.

Control:

- IDLE: responsible for initialization and loading. divreq_rdy=1 indicates new requests can be accepted. When divreq_val=1, inputs are converted into abs_a, abs_b and loaded into registers, and counter is cleared; if in signed mode, then latch div_sign and rem_sign. After completion, FSM transfers to RUN.
- RUN: responsible for executing the division algorithm. Each iteration: a_reg shifts left by one, and perform Diff = A - B. If Diff >= 0, quotient bit is set to 1. If Diff < 0, quotient bit is set to 0. counter ← counter + 1. When ctr_done=1, after 32 iterations, FSM transfers to RESP.
- RESP: responsible for outputting the final result. Quotient and remainder are restored with two's complement through div_sign_mux_sel and reg_sign_mux_sel. divresp_val=1 indicates the result is valid. When divresp_rdy=1, FSM returns to IDLE, ready to process the next request.

(3) Modified Booth Algorithm

Datapath:

- Since the number of iterations is 16, a 4-bit counter is required. Ctr_done is used to determine whether the iteration is completed.
- a_reg[63:0]: shifted left by 2 bits each iteration, a_shift_out = a_reg << 2.
- b_reg[32:0]: shifted right by 2 bits each iteration, b_shift_out = b_reg >> 2.
- Since Modified Booth decides among five operations 0, +A, +2A, −A, −2A according to b_reg[2:0], I use case(b_reg[2:0]) to generate booth_add, and update the partial sum with result_reg ← result_reg + booth_add.

Control:

- IDLE**:** same as iterative multiplier.
- RUN: mainly responsible for the algorithm operation. Each iteration shifts a_reg left by 2 bits, b_reg right by 2 bits, and increments the counter. According to b_reg[2:0], decide the addition or subtraction between result_reg and a_reg. When ctr_done=1, meaning 32 iterations are completed, transfer to RESP state.
- RESP: same as iterative multiplier.

(4) Three-Input Multiplier

Datapath:

- Stage0: a_reg[31:0] * b_reg[31:0] = p_reg[63:0]
- Stage1: p_reg[63:0] * c_reg[31:0] = result_reg[95:0]
- Since the number of iterations is 16, a 4-bit counter is required. Ctr_done is used to determine whether the iteration is completed.
- p_reg[63:0] is first sign-extended to p_base[95:0] to form the Booth operation base.
- Use case(b_reg[2:0]) to generate booth_add, and update the partial sum with result_reg ← result_reg + booth_add. Meanwhile, shift c_reg right by two bits.

Control:

- IDLE: After receiving the input, first execute Stage0 (a*b), latch {P, c} into s0_reg, and then move it to the Stage1 buffer. The state transitions to RUN.
- RUN**:** Perform the Booth operation of Stage1.
- RESP**:** Mainly responsible for outputting the final 96-bit result. When muldivresp_rdy=1, it means the result is received. FSM returns to IDLE, waiting for the next request.

PART3 *Testing Methodology*

- Iterative Multiplier corner case:
  0x7fffffff * 0x7fffffff (maximum positive number), 0x80000000 * 0x80000000 (minimum negative number squared), check whether there is overflow.

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=mul +a=7fffffff +b=7fffffff
./imuldiv-iterative-sim +op=mul +a=80000000 +b=80000000
VCD info: dumpfile dump.vcd opened for output.
0x7fffffff * 0x7fffffff = 0x3fffffff00000001
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
VCD info: dumpfile dump.vcd opened for output.
0x80000000 * 0x80000000 = 0x4000000000000000
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

7fffffff × 7fffffff = 3FFFFFFF00000001

完整計算:
7fffffff = 2147483647(Base 10)
7fffffff = 2147483647(Base 10)
2147483647 × 2147483647 = 4611686014132420609(Base 10) = **3FFFFFFF00000001(Hex)**

80000000 × 80000000 = 4000000000000000

完整計算:
80000000 = 2147483648(Base 10)
80000000 = 2147483648(Base 10)
2147483648 × 2147483648 = 4611686018427387904(Base 10) = **4000000000000000(Hex)**

- Satisfy actual result
- Iterative Divider corner case:
  (0x80000000/(-1)): MIN_INT / (-1)

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=div +a=80000000 +b=ffffffff
VCD info: dumpfile dump.vcd opened for output.
0x80000000 / 0xffffffff = 0x80000000
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

- From the figure, it can be seen that it matches the actual result.

PART4 Evaluation

- 0xbadbeeef*0x10000000=0xfbadbeeef0000000

  (a)For Iterative Multiplier

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=mul +a=badbeeef +b=10000000
VCD info: dumpfile dump.vcd opened for output.
0xbadbeeef * 0x10000000 = 0xfbadbeeef0000000
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

  (b)For Modified Booth Algorithm

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-booth-sim +op=mul +a=badbeeef +b=10000000
VCD info: dumpfile dump.vcd opened for output.
0xbadbeeef * 0x10000000 = 0xfbadbeeef0000000
Cycle Count =         17
../imuldiv/imuldiv-booth-sim.v:121: $finish called at 185 (1s)
```

- 0xf5fe4fbc/0x00004eb6=0xffffdf75

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=div +a=f5fe4fbc +b=00004eb6
VCD info: dumpfile dump.vcd opened for output.
0xf5fe4fbc / 0x00004eb6 = 0xffffdf75
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

- 0x08a22334%0xfdcba02b=0x020503b5

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=rem +a=08a22334 +b=fdcba02b
VCD info: dumpfile dump.vcd opened for output.
0x08a22334 % 0xfdcba02b = 0x020503b5
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

- 0xf5fe4fbc/ u0x00004eb6=0x00032012

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=divu +a=f5fe4fbc +b=00004eb6
VCD info: dumpfile dump.vcd opened for output.
0xf5fe4fbc /u 0x00004eb6 = 0x00032012
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

- 0xa56adca% u0xfabc1234=0xa56adca

```
qiu@LAPTOP-BVJ16IMQ:~/lab1/build$ ./imuldiv-iterative-sim +op=remu +a=0a56adca +b=fabc1234
VCD info: dumpfile dump.vcd opened for output.
0x0a56adca %u 0xfabc1234 = 0x0a56adca
Cycle Count =         33
../imuldiv/imuldiv-iterative-sim.v:130: $finish called at 345 (1s)
```

PART5 Conclusion

In this experiment, the multiply/divide unit requires 33 clock cycles for all operations, meeting the performance requirement; while the modified Booth algorithm multiplier requires only 17 clock cycles, which can be inferred as shifting two bits per iteration, thus halving the number of iterations needed. The performance is significantly better than the iterative multiplier without losing correctness. In PART4, both signed and unsigned tests were also conducted, and the obtained results matched the actual values, proving that the design of the computation unit in this experiment is correct and reliable